IRWA Final Project — Part 1: Text Processing and Exploratory Data Analysis

Oct 22, 2026 · @Xavi

Information Retrieval and Web Analytics (IRWA), 2026 — Universitat Pompeu Fabra.

Student	ID
Xavier Gutiérrez	uXXXXXX
[Name 2]	uXXXXXX
[Name 3]	uXXXXXX

- **GitHub repository:** github.com/XaviGuti25/irwa-search-engine-2026
- **Tag:** `IRWA-2026-part-1`
- **Code:** `project_progress/part_1/part1.ipynb`

1. Introduction

Part 1 prepares the corpus for the search engine built in the next parts and describes it through an exploratory analysis. The corpus is an e-commerce fashion products dataset with **28,080 products** and 17 fields per product (title, description, brand, category, prices, rating, product details, seller, URL, etc.). Every `pid` is unique.

All the code is in one Jupyter notebook written in Python 3. It uses NLTK (as suggested in the statement) for tokenization, stop words, stemming and entity recognition, pandas and NumPy for data handling, and Matplotlib and `wordcloud` for the plots. The README explains how to run it.

2. Text pre-processing (title and description)

We implemented a single function, `preprocess(text)`, that turns a string into a list of terms. It is applied to the title, the description and the other text fields we index. The same function will be applied to the queries in Parts 2 and 3, so that query terms and index terms always match.

The steps, in order:

Step	What it does	Why
Remove non-ASCII characters	Drops emojis and special symbols	Seller-written descriptions contain symbols that are not useful terms
Lowercasing	`Cotton` → `cotton`	The same word must be one term
Remove possessive *'s*	`women's` → `women`	Gender words are key in the queries
Normalise *t-shirt*	`t-shirt`, `t shirt`, `tshirts` → `tshirt`	The same product type is written in three ways
Punctuation removal	Punctuation replaced by spaces	Required; also splits glued words like `look.Designed` → `look designed`, very frequent in the scraped text
Tokenization	NLTK `word_tokenize`	Required
Stop-word removal	NLTK English stop-word list	Required; these words have no topical meaning and a very low IDF
Remove tokens with digits and 1-letter tokens	Drops `1005combo2`, `100`, `s`	Style codes and quantities are never query terms; numeric information is kept in numeric fields instead (Section 4)
Stemming	NLTK Porter stemmer: `printed` → `print`, `sleeves` → `sleev`	Required; groups variants of the same word

The extra steps (non-ASCII removal, possessives, *t-shirt* normalisation and digit-token removal) are our additions to the required ones. *men* and *women* are not in the stop-word list, so they are kept. This matters because both validation queries depend on gender.

Example from the first product:

- **Raw:** *Yorker trackpants made from 100% rich combed cotton giving it a rich look.Designed for Comfort,Skin friendly fabric…*
- **Processed:** `yorker, trackpant, made, rich, comb, cotton, give, rich, look, design, comfort, skin, friendli, fabric, …`
- **Query 1** *women full sleeve sweatshirt cotton* → `women, full, sleev, sweatshirt, cotton`

- **Query 2** *men slim jeans blue* → `men, slim, jean, blue`

We chose stemming over lemmatization because it is what the statement asks for and it is fast on 28,080 documents. The cost is that some stems are not real words (`friendli` , `sleev`); this does not matter for matching, because documents and queries are stemmed in the same way.

3. Output fields and categorical fields

3.1 Fields returned for each document

The 14 fields that must be returned (`pid, title, description, brand, category, sub_category, product_details, seller, out_of_stock, selling_price, discount, actual_price, average_rating, url`) are kept **unchanged** in the corpus so they can be shown in the results. The processed terms are stored in **new** columns (`title_tokens` , `description_tokens` , `details_tokens` and the combined `tokens`). `pid` is never modified because it is the key used for evaluation.

3.2 How to handle category, sub_category, brand, product_details and seller

Approach	Pros	Cons
Merge all fields into one text field	Simple, a single index; a query term matches wherever it appears	Field importance is lost (a match in the title counts the same as one in a long description); noisy fields dilute term frequencies and make documents longer
Index each field separately	Allows field weighting (e.g. boost the title) and exact filters (brand, sub_category)	Bigger, more complex index and scoring; the weights must be tuned

Our decision is a hybrid. We build one combined searchable field with `title + description + brand + sub_category + product_details` (informative values only), which will be indexed in Part 2. We also keep every field separately, so that in Part 3 we can weight them differently or use them as filters.

What we did with each field, and why:

- **category — not added to the text.** 96.6% of products are *Clothing and Accessories*, 3.3% *Footwear*, and the rest is almost empty. A term present in almost every document has an IDF close to 0, so it cannot help ranking. We keep it for filtering.

- **sub_category — added.** It is more distinctive (24 values: *Topwear*, *Bottomwear*, *Winter Wear*…). Some values have a seller prefix (e.g. *Roy Clothing and Accessories*, *SUNSHOPPING Bags, Wallets & Belts*), so we map them to their parent value (123 products).
- **brand — added, and kept separately.** It is very useful when the query names a brand, and it can be used as a filter. 7.2% of products have no brand.
- **product_details — added (values of informative keys).** It is a list of key/value pairs with 127 different keys. It contains exactly the attributes users search for (*Fabric: Cotton*, *Sleeve: Full Sleeve*, *Ideal For: Women*, *Fit: Slim*, *Color: Blue*). Also, 39.7% of products have no description, so for many products the details are the only descriptive text besides the title. We drop keys that are noise for search: *Style Code, Fabric Care, Sales Package, Country of Origin, Pack of, Reversible, Size, Other Details, Model Name, Suitable For, Brand Color, Brand Fit, Sleeve Type*. Only the values are indexed, not the keys (users type *cotton*, not *fabric cotton*).
- **seller — not added to the text.** Seller names (*e.g. Shyam Enterprises*) are not what users look for in product queries and would create false matches. We keep them as metadata to display.

4. Numeric and boolean fields

These fields are not indexed as textual terms. A token like `921` or `69%` never matches the words of a query, it would only add noise to the vocabulary, and conditions such as *under ₹1000* or *rating of at least 4* cannot be answered with term matching. Instead, they are converted into numeric types, to be used in Part 3 as:

- **Filters:** price range, minimum rating, hide out-of-stock products.
- **Ranking signals:** boost products with a high rating or discount, demote out-of-stock products, break ties between documents with the same text score.

In the raw data all of them except `out_of_stock` are strings. The conversion rules:

Field	Raw example	Converted	Missing values
`selling_price`, `actual_price`	`"2,999"`	`2999.0` (float)	`NaN`
`discount`	`"69% off"`	`69.0` (percentage)	`NaN`
`average_rating`	`"3.9"`	`3.9` (float)	`NaN`
`out_of_stock`	`false`	`False` (boolean)	none

A missing rating is stored as `NaN` and not as 0, because a product without reviews is not a badly rated product. As a sanity check, the discount matches `1 - selling_price / actual_price` (±1 point) in 99.97% of the products that have both prices, and the selling price is never higher than the actual price.

5. Reflection with validation_labels.csv

The validation labels confirm that the title is the most discriminative field and that product_details must be indexed. The file has 40 labelled documents: query 1 (*women full sleeve sweatshirt cotton*) has 13 relevant and 7 non-relevant, and query 2 (*men slim jeans blue*) has 10 and 10. For each field we measured the average share of the (processed) query terms that appear in it:

Query	Field	Non-relevant (0)	Relevant (1)
1	title	0.46	0.80
1	description	0.63	0.57
1	product_details	0.57	0.57
1	combined	1.00	0.98
2	title	0.72	0.80
2	description	0.40	0.35
2	product_details	0.45	0.62
2	combined	0.80	0.90

What we learned:

- **The title separates relevant from non-relevant documents best.** For query 1, relevant documents match 80% of the query terms in the title against 46% for non-relevant ones. The non-relevant ones include women's *half sleeve T-shirts*: they match *women* and *cotton* but not *sweatshirt* or *full sleeve*. This suggests giving the title a higher weight in Part 3.
- **product_details is necessary.** For query 1, *cotton* never appears in the titles; it only appears in the *Fabric* detail (*Cotton Blend*, *Pure Cotton*). Without this field those documents could not match the full query.
- **The combined field alone does not separate the classes.** For query 1, non-relevant documents match all the query terms in the combined field (1.00). So a plain TF-IDF over one merged field will not be enough, and field weighting is justified.

- **Gender is decisive.** For query 2, 8 of the 10 non-relevant documents are women's jeans. Keeping *men* and *women* as terms is essential.
- **Some labels look inconsistent.** Products with the same title, brand and fabric appear with different labels, so no pre-processing can reach a perfect score on this set.

6. Exploratory data analysis

The corpus is made of short, repetitive product texts with many missing descriptions, and products that are cheap, heavily discounted and mostly in stock. The plots for each point are in the notebook (sections 2.1–2.8).

6.1 Missing values

Field	Missing	%
description	11,150	39.7
average_rating	2,261	8.1
brand	2,009	7.2
seller	1,643	5.9
discount	855	3.0
actual_price	777	2.8
selling_price	2	0.0

Title, category, sub_category, product_details, pid and url are never empty. Since almost 40% of products have no description, retrieval cannot rely on it alone.

6.2 Text statistics and vocabulary

- **Word counts:** a product has on average 40.7 words (title + description) before pre-processing and 24.5 terms after it. The distribution is very skewed: many products have only the title, while a few descriptions reach almost 600 words.
- **Titles** are short, 7.1 words on average, and very repetitive: there are only 5,953 distinct titles, so 78.8% of the products share their title with another product. The most common are *Slides* (544 products) and *Printed Men Round Neck Black T-Shirt* (418). Many documents will therefore get the same score from their title, and other signals will be needed to break ties.
- **Sentences:** descriptions have 3.1 sentences on average, with 17.8 words per sentence.

- **Most frequent words** (after pre-processing, without stemming): in titles *women*, *men*, *tshirt*, *neck*, *solid*, *round*, *printed*; in descriptions *tshirt*, *cotton*, *wear*, *casual*, *made*, *fit*, *look*. Titles describe the product type and gender; descriptions are more about material and style. The word clouds in the notebook show the same picture.

Vocabulary size of title + description at each step:

Stage	Vocabulary	Total tokens
Raw (lowercase, split on spaces)	10,606	1,017,425
Pre-processed, no stemming	6,124	686,812
Pre-processed + stemming	4,654	686,812
Combined indexed field (all text fields)	5,058	1,071,229

Pre-processing cuts the vocabulary by 56% and the number of tokens by 32%. Cleaning (punctuation, digit tokens and stop words) removes 42% of the terms; stemming removes another 24% of the remaining ones.

6.3 Categories

Clothing and Accessories covers 96.6% of the products. Within sub_category, *Topwear* dominates (54.5%), followed by *Bottomwear* (13.1%) and *Winter Wear* (8.9%).

6.4 Prices, discounts and ratings

Measure	Selling price (₹)	Actual price (₹)	Discount (%)	Rating
Mean	706	1,456	50.3	3.63
Median	545	1,199	53	3.8
Min – max	99 – 7,999	150 – 12,999	1 – 87	1.0 – 5.0

- Most products cost between ₹390 and ₹820, and half of them have a discount of 53% or more.
- Ratings are concentrated between 3.2 and 4.1.
- Rating is practically uncorrelated with price and discount (Spearman −0.03 and −0.10), so a cheaper or more discounted product is not rated better. Selling price and discount are negatively correlated (−0.37).

- **Rankings** (tables in the notebook): the top-rated products have a rating of 5.0, but the dataset has no number of reviews, so a 5.0 can come from very few reviews. The most expensive product is a women's track suit at ₹7,999; the cheapest products are socks and a bandana at around ₹99; the highest discount is 87%, on women's T-shirts.

6.5 Brands and sellers

There are 324 brands and 534 sellers. The brands with the most products are *ECKO Unl* (951), *Free Authori* (860), *ARBO* (806), *REEB* (802) and *Pu* (798). The top sellers are *RetailNet* (1,411), *SandSMarketing* (887) and *BioworldMerchandising* (842). Among brands with at least 100 products, the best rated are *PixF* (4.46) and *Cots* (4.44). Many brand names look truncated (*ECKO Unl*, *Pu*, *REEB*), which would hurt queries that write the full brand name.

6.6 Stock availability

94.1% of products are in stock and 5.9% (1,644) are out of stock. The highest share of out-of-stock products is in *Clothing Accessories* (16.9%), followed by *Winter Wear* (9.5%).

6.7 Entity recognition

We ran the NLTK named-entity chunker on 500 distinct descriptions. It found 1,064 PERSON, 514 ORGANIZATION and 334 GPE mentions. The model is trained on news text, so on product descriptions it is noisy: words like *Cotton* or *Full Sleeve* are tagged as PERSON. It does detect some brands (*ECKO*, *Free Authority*) and places (*India*). Because of this noise, we do not use entities as a feature in the index.

7. Assumptions and limitations

- **Queries are in English** and will be processed with the same `preprocess` function as the documents.
- **Numbers in the text are not searchable.** Removing tokens with digits loses information such as *pack of 2* or *3/4 sleeve*. We accept this because that information is either in `product_details` or in the numeric fields.
- **Missing numeric values stay missing** (`NaN`); they are not replaced by 0 or by the mean. How to treat them in ranking (e.g. products without rating) will be decided in Part 3.
- **The list of dropped product_details keys** was chosen by inspecting the most frequent keys and their values. Less frequent keys are kept.
- **Stemming errors are accepted.** The Porter stemmer sometimes over-stems or produces non-words, but documents and queries are affected equally.

- **Data quality** limits what retrieval can achieve: some brand names are truncated, many titles are identical, and some validation labels are inconsistent.
- **Entity recognition** was run on a random sample of 500 descriptions (`random_state=42`) because the NLTK chunker is slow, so its counts are indicative only.

8. Use of AI tools

We used Claude (Anthropic) to help set up the GitHub repository, to brainstorm pre-processing and field-handling strategies, to help write and debug the notebook code, and to draft a first version of this report from our notebook results. We reviewed, ran and modified all the code, checked every number in the report against the notebook outputs, and rewrote the explanations in our own words. We understand and can explain all the submitted code and decisions.

[Adjust this paragraph so it describes exactly how your group used AI.]